

Git 手動操作手冊

對象：需要在本專案手動用 Git 的人（不靠 IDE 按鈕，用指令操作）。**環境：**Windows + PowerShell、Unity 專案。**目標：**看完能安全地「存檔（commit）→ 上傳（push）→ 拉新（pull）→ 出事時救回來」。

如果你只想快速上手，先看「§1 心智模型」+「§4 日常流程」+「§11 速查表」三節即可，其餘當字典查。

1. 心智模型（最重要，先懂這個）

Git 把你的檔案分成**四個區域**，所有指令都是在這四區之間搬資料：

工作區 (Working Tree)	暫存區 (Staging/Index)	本機儲存庫 (Local Repo)	遠端儲存庫 (Remote, 如 GitHub)
你正在編輯的檔	→ git add →	git commit →	git push
			← git pull / git fetch

- **工作區：**你眼前實際的檔案，編輯就是改這裡。
- **暫存區：**你「挑選」這次要記錄哪些變更（`git add`）。可以只存一部分。
- **本機儲存庫：**`git commit` 把暫存的內容變成一個永久快照（一個 commit）。
- **遠端儲存庫：**別人也看得到的雲端版本，用 `git push` 上傳、`git pull` 下載。

一個 **commit** = 一張「存檔照片」，有唯一編號（hash，如 `87bccfb`）、作者、時間、訊息、以及和上一張的差異。

2. 名詞對照表

名詞	白話解釋
repository (repo · 儲存庫)	被 Git 追蹤的整個專案資料夾
commit	一次存檔 (快照) + 一段說明訊息
branch (分支)	一條獨立的開發線，可平行進行不互相干擾
HEAD	「你現在站在哪個 commit / 分支」的指標
staged (已暫存)	已被 <code>git add</code> 、下次 commit 會收進去的變更
tracked / untracked	檔案是否已被 Git 追蹤；新檔預設是 untracked
remote (遠端)	雲端的 repo，預設名稱通常叫 <code>origin</code>
clone	把遠端 repo 完整複製到本機
merge / rebase	把兩條分支的內容合在一起的兩種方式
conflict (衝突)	兩邊改了同一處，Git 無法自動決定，要人工處理

3. 安裝與初次設定 (每台電腦只需一次)

```
git --version # 確認已安裝
git config --global user.name "你的名字"
git config --global user.email "你的信箱"
git config --global core.autocrlf true # Windows 建議：自動處理換行符
```

- `--global` = 對這台電腦的所有 repo 生效。
- 本專案已經是 Git repo (根目錄有 `.git/`)，不需要再 `git init`。

4. 日常流程 (90% 的時間只用這幾個)

```
git status          # 我改了什麼？哪些已暫存、哪些還沒？(最常用)
git add <檔案>      # 把指定檔案加入暫存區
git add -A          # 把「所有」變更(含新檔/刪檔)加入暫存區
git commit -m "訊息" # 把暫存區內容存成一個 commit
git log --oneline -10 # 看最近 10 筆 commit
```

標準一輪：1. `git status` 看現況 2. `git add -A` (或挑特定檔) 3. `git commit -m "儲存點：做了什麼"` 4. (要同步雲端時) `git push`

寫 commit 訊息的小原則：寫**為什麼 / 做了什麼**，而不是流水帳。本專案習慣用「儲存點：...」開頭，可沿用以保持一致。

5. ⚠ 本專案環境地雷 (一定要看)

這些是我們實際踩過的坑：

5-1. PowerShell 不支援 `&&`

Bash 的 `git add . && git commit ...` 在 PowerShell 會報錯。請改用：

```
git add -A ; git commit -m "訊息" # 用分號分隔
```

或乾脆分兩行各打一次。

5-2. 多行 / 中文 commit 訊息：用檔案

PowerShell 沒有 `<<EOF` heredoc。要寫多行訊息時，先寫進一個檔案再用 `-F`：

```
# 把訊息寫進 msg.txt (可多行)，然後：
git commit -F msg.txt
del msg.txt
```

單行中文訊息用 `-m "儲存點：..."` 即可。

5-3. 不要把建置產物提交進去

Unity / IDE 會自動產生大量暫存檔，**不該進版**。本專案 `.gitignore` 已排除：- `Library/`、`Temp/`、`Obj/`、`Build/`、`Logs/` - `.utmp/`、`.vs/`、`CardGame-`

debug_BurstDebugInformation_DoNotShip/ - *.log 、 UpgradeLog*.htm 、 *.csproj 、 *.sln
等

若 `git status` 冒出一堆不認識的產物，先確認它們有沒有被 `.gitignore` 蓋到，不要急著 `git add -A` 。

6. Unity 專案的特別注意

- `.meta` 檔要一起進版：每個資產（圖、音、prefab）旁邊的 `.meta` 存著 GUID 與匯入設定，必須和資產一起 commit，否則別人開啟會出錯。`.gitignore` 已設定保留 `Assets/**/*.*meta` 。
- 場景 / prefab 容易衝突：`.unity`、`.prefab` 是 YAML，多人同時改同一個場景常會衝突且難手動合併。原則：同一個場景盡量別兩人同時改，改前先 `git pull` 。
- 存檔資料別進版：玩家個人存檔（如 `Assets/PlayerDataSnapshots/*.csv`）已被忽略，避免把個人測試資料推給別人。

7. 看歷史與差異

```
git log --oneline --graph -20      # 圖形化看分支與歷史
git show <hash>                    # 看某個 commit 的完整內容
git diff                           # 工作區 vs 暫存區 (還沒 add 的變更)
git diff --staged                  # 暫存區 vs 上一個 commit (已 add 的變更)
git diff <hashA> <hashB>           # 比較兩個 commit
git blame <檔案>                   # 每一行最後是誰、哪個 commit 改的
```

8. 分支與合併

```
git branch                         # 列出本機分支 (* 是目前所在)
git switch -c feature/新功能       # 建立並切換到新分支
git switch main                     # 切回 main
git merge feature/新功能            # 把該分支合併進目前分支
git branch -d feature/新功能        # 合併後刪掉分支
```

為什麼用分支：在獨立分支做實驗或新功能，做壞了也不影響 `main`，完成再合併。

9. 遠端同步 (push / pull / clone)

```
git clone <網址>          # 第一次把遠端 repo 抓下來
git remote -v             # 看遠端位址 ( 通常叫 origin )
git pull                  # 抓遠端最新並合併到目前分支 ( 動手前先做 )
git fetch                 # 只抓遠端更新、先不合併 ( 較保守 )
git push                  # 把本機 commit 上傳
git push -u origin <分支名> # 第一次推新分支 ( 建立追蹤關係 )
```

養成習慣：開始工作前先 `git pull`，減少衝突。

10. 還原與救援 (出事別慌，幾乎都救得回來)

```
# 還沒 commit，想丟棄某檔的修改 ( 回到上一個 commit 的樣子 )
git restore <檔案>

# 已 add 但想取消暫存 ( 把檔案移出暫存區，內容保留 )
git restore --staged <檔案>

# 暫時收起目前未完成的修改去做別的事，之後再拿回來
git stash          # 收起來
git stash pop      # 拿回來

# 修改剛剛那一個 commit 的訊息或補檔 ( 注意：尚未 push 才安全 )
git commit --amend

# 想「撤銷某個已 push 的 commit」但保留歷史 ( 最安全的反悔方式 )
git revert <hash>    # 會新增一個「相反」的 commit
```

找回看似消失的 commit：

```
git reflog          # Git 記錄你 HEAD 的每一步移動，幾乎所有東西都能從這裡找回
```

11. ⚠️ 危險操作（理解後再用）

指令	風險	說明
<code>git reset --hard</code>	會丟掉未提交的修改	把工作區強制重置，未存檔的編輯永久消失
<code>git push --force</code>	會覆蓋遠端歷史	可能蓋掉別人的 commit；對 <code>main</code> 尤其危險
<code>git clean -fd</code>	刪除未追蹤檔案	連同你還沒 add 的新檔一起刪
<code>git rebase</code>	改寫歷史	在已 push 的共用分支上 rebase 會造成混亂

原則：不確定的指令先在不重要的分支試，或先 `git status` / `git log` 確認狀態。已經 push 到共用分支的東西，用 `git revert` 反悔，不要用 `reset --hard` + force push。

12. 常見情境速查表

我想做什麼	指令
看現在改了什麼	<code>git status</code>
存檔（全部變更）	<code>git add -A</code> ； <code>git commit -m "訊息"</code>
只存某個檔	<code>git add 路徑/檔案</code> ； <code>git commit -m "訊息"</code>
上傳到雲端	<code>git push</code>
抓雲端最新	<code>git pull</code>
丟棄某檔還沒存的修改	<code>git restore 檔案</code>
取消剛剛的 add	<code>git restore --staged 檔案</code>
改剛剛的 commit 訊息（未 push）	<code>git commit --amend</code>
反悔某個已 push 的 commit	<code>git revert <hash></code>
暫時收起手邊修改	<code>git stash</code> → <code>git stash pop</code>
看歷史	<code>git log --oneline --graph -20</code>
找回弄丟的 commit	<code>git reflog</code>

13. 一句話總結

改檔 → `git status` 看清楚 → `git add` 挑要存的 → `git commit` 存檔 → `git push` 上雲。出事先 `git status` / `git reflog`，別急著用 `--hard` 或 `--force`。

文件位置：`GIT_使用手冊.md`（專案根目錄）。如有專案特定流程更新，直接編修本檔。